



汇编语言

3 数据表示与存储

张青

ieqzhang@zzu.edu.cn

教学内容



- 复习数据表示

教材章节：2.1-2.3

慕课：3.1-3.4,3.6

- 多字节数据存储

- 地址对齐

教材章节：2.4

慕课：3.8-3.9

- x86-64处理器导学

教材章节：3.1-3.4

慕课：4.1-4.4

- ▶ 数制、编码

- ▶ GAS变量定义伪指令

- ▶ 常量定义伪指令

- ▶ 大端方式与小端方式

- ▶ 地址对齐及伪指令

- ▶ 列表文件

- ▶ 寄存器

- ▶ 工作方式与存储模型

- ▶ 逻辑地址与物理地址转换

本周学习目标



- 能进行不同数制及编码之间的转换
- 能阐述C语言数据类型与汇编语言变量之间的关系
- 能用GAS伪指令定义变量、常量、分配空间、地址对齐
- 能对比大端方式与小端方式
- 会分析数据在内存中的存储及内存中存储内容对应的数据
- 会用GAS生成列表文件并分析列表文件的内容
- 养成细致严谨的态度，注意C语言与汇编语言数据类型之间的关系
- 结合汇编代码分析深入理解组成原理中数据表示，培养系统观

学习目标



- **记忆**伪指令的名字、功能；多字节数据在存储器中的存储方式；记忆数制转换、补码转换，BCD码，有符号数和无符号数的定义；常用的ASCII码
- **能够**计算有符号数和无符号数的表示范围；将数值或字符、字符串转换成对应的二进制编码；正确使用伪指令定义变量；分析不同编码的二进制数表示范围；在汇编语言程序中正确书写常数；分析多字节数据在内存中的存储；使用变量定位、对齐、地址和类型伪指令；结合列表文件分析数据的表示和存储；结合数据表示和存储分析C语言程序
- 养成良好的编程习惯，养成细心、严谨的工作态度，养成团队合作，互相帮助的精神



邀请码: 12570104

APP首页右上角输入



该邀请码2026年08月26日前有效

计算机类2024级3班4班

复习数据表示

随堂练习(学习通): 系统概述随堂练习

邀请码: 97974770

APP首页右上角输入



复习数据表示

随堂练习(学习通): 系统概述随堂练习



该邀请码2026年08月26日前有效

计算机类2024级9班10班

0x81323334的多视角解码

课堂活动



编码类型	解释方法	真值
无符号数	直接转换	
原码	最高位符号，其余为绝对值	
反码	符号位不变，其余取反	
补码	符号位不变，取反+1	
BCD码	每4位转十进制	
IEEE 754	s/e/M分段计算	-3.274×10^{-38}
ASCII	每字节转字符	
核心启示： • 相同二进制，不同解释 → 不同含义 • 数据类型决定解码方式		

机器数多重解释：0x81323334的多视角解码



编码类型	解释方法	真值
无符号数	直接转换	2,167,550,772
原码	最高位符号，其余为绝对值	-20,040,244
反码	符号位不变，其余取反	-2,127,443,403
补码	符号位不变，取反+1	-2,127,443,404
BCD码	每4位转十进制	81,323,334
IEEE 754	s/e/M分段计算	-3.274×10^{-38}
ASCII	每字节转字符	"?234" (不可显示)
核心启示： • 相同二进制，不同解释 → 不同含义 • 数据类型决定解码方式		

伪指令	大小	C类型	示例
	1字节	char	.byte 65 → 'A'
	2字节	short	.word -1 → 0xFFFF
	4字节	int	.long -1 → 0xFFFFFFFF
	8字节	long/指针	.quad str → 地址
	N字节	char[]	.ascii "AB\0"

伪指令	大小	C类型	示例
.byte	1字节	char	.byte 65 → 'A'
.word	2字节	short	.word -1 → 0xFFFF
.long	4字节	int	.long -1 → 0xFFFFFFFF
.quad	8字节	long/指针	.quad str → 地址
.ascii	N字节	char[]	.ascii "AB\0"

例2-14: eg0214.c



```
int x = -1;
unsigned int u = 2147483648;
long l = -1;
unsigned long ul = -1;
short s = -1;
unsigned short us = -1;
char c = 'A';
unsigned char uc = 200;
char str[] = "Hello Assembly!";
char str1[] = {'C','h','i','n','a','\0'};
char *ps = str;
```

eg0214.c对应的汇编语言程序 课堂活动



提问：如何将eg0214.c 转换为汇编语言程序eg0214.S?

```
int x = -1;
unsigned int u = 2147483648;
long l = -1;
unsigned long ul = -1;
short s = -1;
unsigned short us = -1;
char c = 'A';
unsigned char uc = 200;
char str[] = "Hello Assembly!";
char str1[] = {'C','h','i','n','a','\0'};
char *ps = str;
```

x:	?	-1
u:	?	-2147483648
l:	?	-1
ul:	?	-1
s:	?	-1
us:	?	-1
c:	?	65
uc:	?	-56
str1:	?	"China\0"
str:	?	"Hello Assembly! "
ps:	?	str

eg0214.c对应的汇编语言程序



提问：如何将eg0214.c 转换为汇编语言程序eg0214.S?

```
int x = -1;
unsigned int u = 2147483648;
long l = -1;
unsigned long ul = -1;
short s = -1;
unsigned short us = -1;
char c = 'A';
unsigned char uc = 200;
char str[] = "Hello Assembly!";
char str1[] = {'C','h','i','n','a','\0'};
char *ps = str;
```

x:	.long	-1
u:	.long	-2147483648
l:	.long	-1
ul:	.long	-1
s:	.word	-1
us:	.word	-1
c:	.byte	65
uc:	.byte	-56
str1:	.ascii	"China\0"
str:	.asciz	"Hello Assembly! "
ps:	.quad	str

例2-14: eg0214.c显示数据



```
printf("x = %x = %d = %u \n", x, x, x);
printf("u = %x = %d = %u \n", u, u, u);
printf("l = %x = %ld = %llu \n", l, l, l);
printf("s = %x = %d = %u \n", s, s, s);
printf("us = %x = %d = %u \n", us, us, us);

printf("c = %x = %d = %u = %c \n", c, c, c, c);
printf("uc = %x = %d = %u = %c \n", uc , uc, uc, uc);
printf("str = %s\n", str);
printf("str1 = %s\n", str1);

printf("ps = %x = %ld", ps);
```

eg0214运行结果:



$x = \text{ffffffff} = -1 = 4294967295$

$u = 80000000 = -2147483648 = 2147483648$

$l = \text{ffffffff} = -1 = 4294967295$

$s = \text{ffffffff} = -1 = 4294967295$

$us = \text{ffff} = 65535 = 65535$

$c = 41 = 65 = 65 = A$

$uc = c8 = 200 = 200 = ?$

$str = \text{Hello Assembly!}$

$str1 = \text{China}$

$ps = 82884020 = 618651224$

列表文件是一种文本文件，含有源程序和对应的目标代码

生成列表文件的方法：

```
as -al=./progs/eg0214.lst -o ./progs/eg0214.o ./progs/eg0214.s
```

```
gcc -Og -masm=intel -Wa,-al -g ./progs/eg0214.c>eg0214.lst
```


eg0214.lst(部分)



行数	偏移地址	目标代码	汇编语句
94	0010	48656C6C 6F204173 73656D62 6C792100	str:
95			.ascii "Hello Assembly!\0"
95			
95			
95			
97	0020	C8	uc:
98			.byte
108	0024	FFFF	s:
109			.word
112	0028	FFFFFFFF	ul:
113			.long
120	0030	00000080	u:
121			.long

常量
*与C语言表示方法相同

-56

-1

-1

-2147483648

```
87                                     ps:
88      0000      10000000      .quad str
88      0000      00000000
90                                     str1:
91      0008      4368696E      .ascii  "China"
91      6100
94                                     str:
95      0010      48656C6C      .ascii  "HelloAssembly!"
95                      6F204173
95                      73656D62
95                      6C792100
97                                     uc:
98      0020      C8           .byte   -56
100                                     c:
101      0021      41           .byte   65
```

104			us:
105	0022	FFFF	.word -1
108			s:
109	0024	FFFF	.word -1
112			ul:
113	0028	FFFFFFFF	.long-1
116			l:
117	002c	FFFFFFFF	.long-1
120			u:
121	0030	00000080	.long -2147483648

常量符号定义

- 常量也可以用符号表示，称为符号常量。

- .equ

- .equ 定义的符号常量不可修改

.equ 符号名, 数值表达式/<字符串>

```
.equ NULL, 0  
message: .ascii "hello, world"  
.equ msglen, .-message
```

- .set

- 可重新赋值

```
.set char, 'A'  
.set char, char+1
```

变量定义伪指令



变量定义伪指令	变量类型	功能
.byte	字节	分配一个或多个字节单元；数据可以是字节量，也可以是字符串常量。 字节量表示8位无符号数或有符号数、字符的ASCII码值
.word	字	分配一个或多个字单元；数据是字量、16位数据。 字量表示16位无符号数或有符号数、16位段选择器、16位偏移地址
.long	双字	分配一个或多个双字单元；数据是双字量、32位数据 双字量表示32位无符号数或有符号数、32位段基地址、32位偏移地址
.quad	四字	分配一个或多个8字节单元；8字节量表示64位数据
.ascii	字符串	为每个字符串中的每个字符分配一个字节单元
.asciz	字符串	为每个字符串中的每个字符分配一个字节单元，以0结尾
.string	字符串	与.asciz功能相同



数据存储

大端方式与小端方式

数据存储的基础规则

- **计算机内存：**数据+代码，以**字或字节**为单位存储数据
- **内存地址：**每个存储单元的唯一地址
- **内存编址方式：**
 - **字节编址（主流）：**每个存储单元存储一个字节
 - 字编址（历史）：每个存储单元存储一个字
- **多字节数据存储方式**
 - 大端方式
 - 小端方式

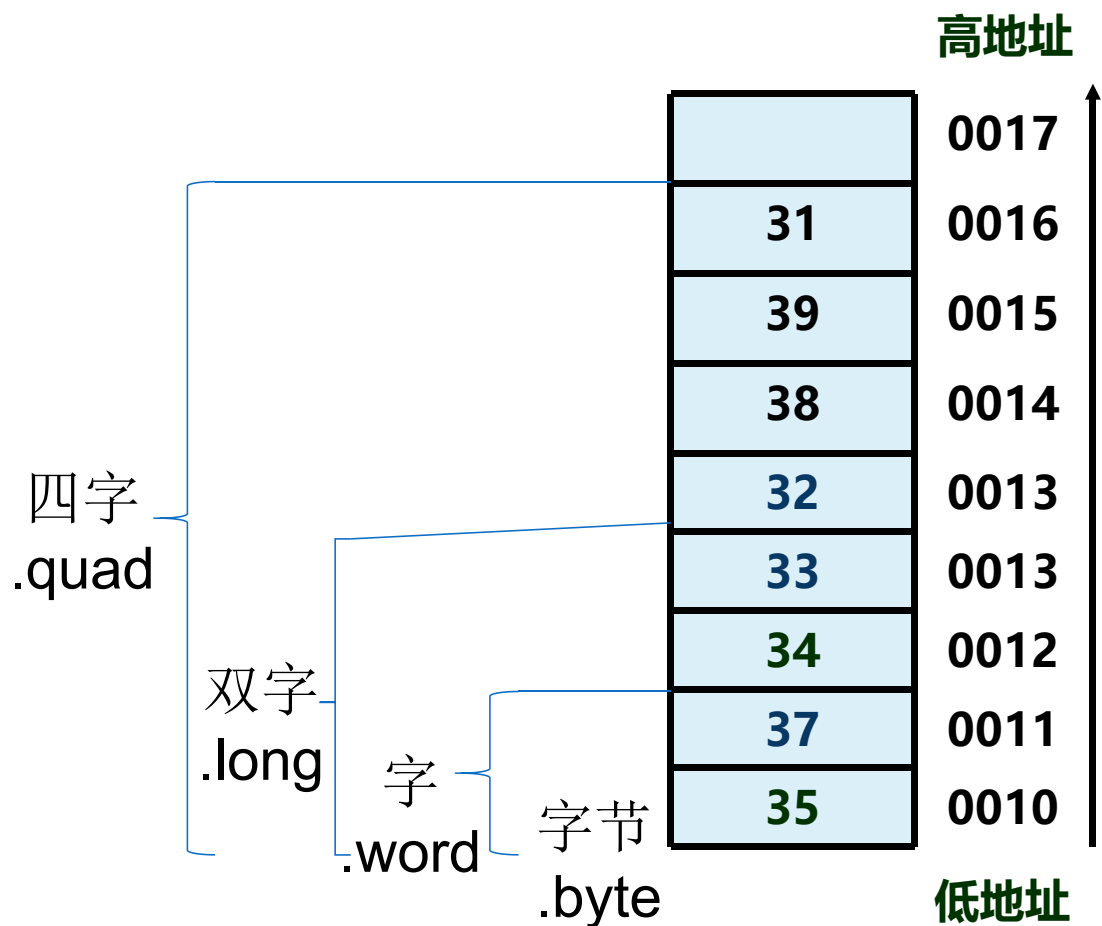
大端方式和小端方式



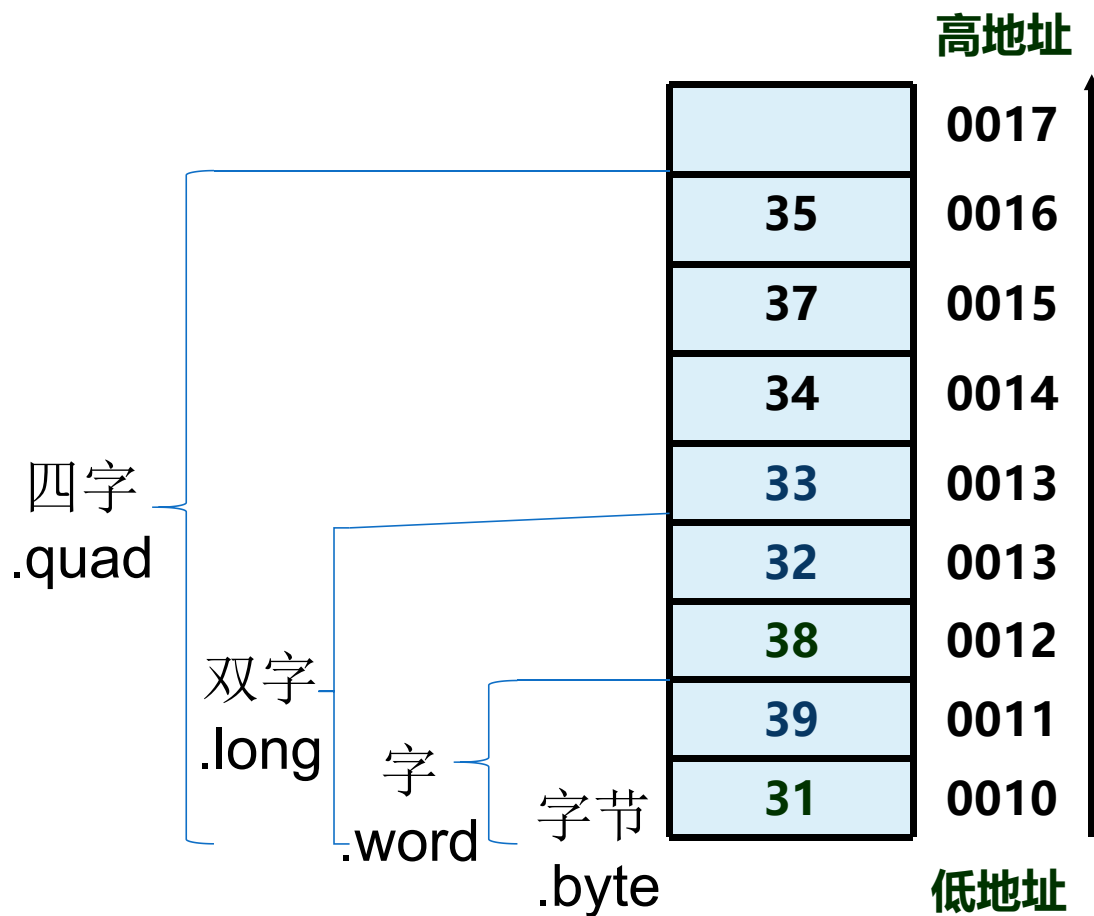
- **小端方式 (Little Endian)** : “**低对低、高对高**” , 即低字节数据存放在低地址存储单元、高字节数据存放在高地址存储单元。
- **大端方式 (Big Endian)** : “**低对高、高对低**” , 即低字节数据存放在高地址存储单元、高字节数据存放在低地址存储单元。
- 无论小端存储还是大端存储, 多字节数据的起始地址都是所占用的最低存储器地址。

数据	字节数	小端方式 低地址到高地址依次显示	大端方式 低地址到高地址依次显示
0x31	1	31	31
0x3139	2	39 31	31 39
0x31393832	4	32 38 39 31	31 39 38 32
0x3139383233343735	8	35 37 34 33 32 38 39 31	31 39 38 32 33 34 37 35

大端方式和小端方式



小端方式



大端方式

主题讨论：变量存储



```
bvar:    .byte  0x39,0x31,0x32,0x38  
wvar:    .word  0x3139,0x3832  
dvar:    .long  0x38323139
```

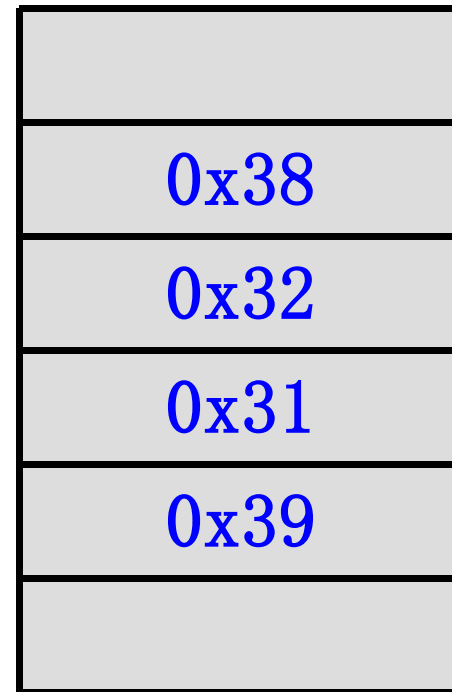


字节量数据的顺序存储



```
bvar:    .byte  0x39,0x31,0x32,0x38
```

高地址 → 03
02
01
00
低地址 → 地址



字量数据的存储顺序

```
wvar:    .word 0x3139,0x3832
```



高地址



0x32

0x38

0x39

0x31

低地址



大端方式

03

02

01

00

地址

0x38

0x32

0x31

0x39

小端方式

双字量数据的存储顺序

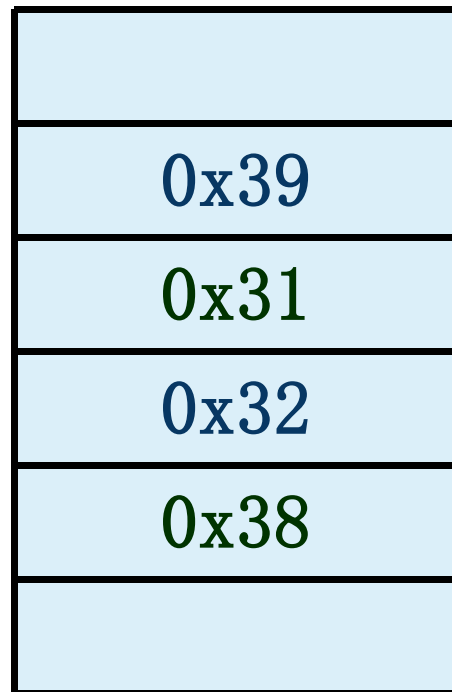
dvar: .long 0x38323139



高地址



低地址



大端方式

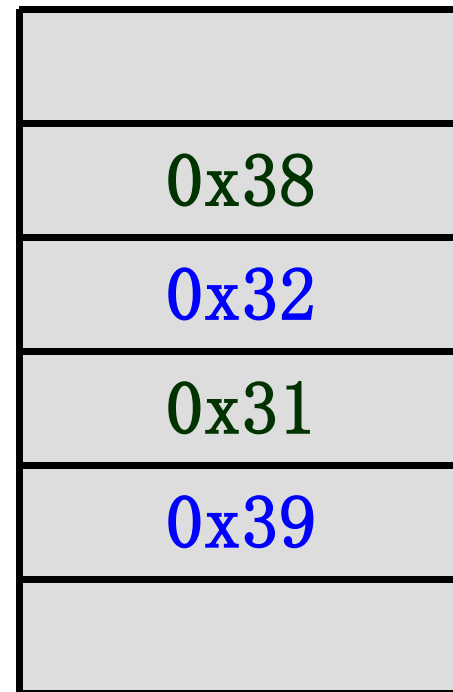
03

02

01

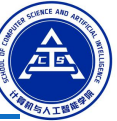
00

地址



小端方式

eg0214.lst(部分)



行数	偏移地址	目标代码	汇编语句
87	0000	10000000 00000000	ps:
88			.quad str
88			
94			str:
95	0010	48656C6C 6F204173 73656D62 6C792100	.ascii "Hello Assembly!\0"
95			
95			
95			
113	0028	FFFFFFFF	.long -1
120	0030	00000080	u:
121			.long -2147483648

类型转换



```
void show_bytes(byte_pointer start, size_t
len) {
    for (size_t i = 0; i < len; i++)
        printf(" %.2x", start[i]);
    printf("\n");
}
```

```
show_bytes((byte_pointer) &i, sizeof(i));
show_bytes((byte_pointer) &l, sizeof(l));
show_bytes((byte_pointer) &c, sizeof(c));
```

```
int i = -12345;
long l;
char c;
l = i;    // int -> long (值不变)
c = (char)i;
```

运行结果展示:

```
c7 cf ff ff
c7 cf ff ff
c7
```



地址对齐

变量定位：对齐地址边界

- N字节数据起始于能够被N整除的地址
 - 2字节、16位数据是被2整除的地址（偶地址）
 - 4字节、32位数据是被4整除的地址（模4地址）
 - 8字节、64位数据是被8整除的地址（模8地址）
- .align 对齐指令
 - 格式： **.align n** # $n \in \{2, 4, 8, \dots\}$ 且 $n \leq$ 当前段对齐属性
 - 功能：将下一数据地址对齐到n的整数倍（地址 % n = 0）

地址A对齐N字节边界

$A \bmod N = 0$ (能够被N整除的地址)

eg0214.lst(部分)



行数	偏移地址	目标代码	汇编语句
101	0021	41	.byte 65
103			.align 2
104			us:
105	0022	FFFF	.word -1
107			.align 2
108			s:
109	0024	FFFF	.word -1
111	0026	0000	.align 4
112			ul:
113	0028	FFFFFFFF	.long -1
			.align 4
120			u:
121	0030	00000080	.long -214748364887

.space 填充指令



- 格式: **.space n [, fill_value]** # fill_value默认为0
- 功能: 分配n字节未初始化空间, 可选填充指定值
- 典型用途: 结构体对齐填充、数组/缓冲区预留空间

```
array: .space 1024    # 分配1KB缓冲区  
pad:   .space 3, 0x90 # 填充3字节, 值=0x90
```

地址对齐：内存对齐问题 (eg0217.c)



```
struct X {  
    int a;  // 4字节 (需4字节对齐)  
    char c; // 1字节  
    int b;  // 4字节 (需4字节对齐)  
};
```

```
struct X x = {-100, 'a', -200};
```

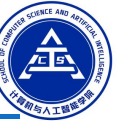
```
int main() {  
    printf("sizeof X = %d \n", sizeof(x));  
    show_bytes((byte_pointer)&x, sizeof(x));  
    return 0;  
}
```

运行结果展示:

sizeof X = 12

9c ff ff ff 61 00 00 00 38 ff ff ff

eg0217.lst(部分)



行数	偏移地址	目标代码	汇编语句
66			.align 8
67			x:
68	0000	9CFFFFFF	.long -100
69	0004	61	.byte 97
70	0005	000000	.space 3
71	0008	38FFFFFF	.long -200
112			ul:
113	0028	FFFFFFFF	.long -1

x86-64处理器导学

教材章节：3.1-3.4

慕课：4.1-4.4

- ▶ 寄存器
- ▶ 工作方式与存储模型
- ▶ 逻辑地址与物理地址转换

教学要点

- 慕课学习：
 - 视频：4.1-4.4
 - 单元测试：3.10
 - 第二章作业：3.11

- ✓ 变量和常量定义伪指令
- ✓ 对齐伪指令
- ✓ 大端方式与小端方式